ECE 470 final Lab Report

Author: Anav | NetID: anavv2
Partner: Susheennder Vijay | NETID: sv74
TA: Jimmy Fang
Section: Monday 9AM (AB5)

15th May 2026

1 Objective

The objective of this lab is to control the UR3 to draw a given image. This is an important building block from the basics of camera control and the forward and inverse kinematics to the industrial applications of such robots. Specifically, usage of Computer vision libraries like OpenCV as have been used in this lab and in the lab 5 serve as foundations to be used in complex industrial assembly, pick-up and place tasks and also in many cases in medical services.

2 Solution

2.1 Identifying strokes, end points and junctions

This section describes the working of the As part of the `draw_image` function. It takes the inputs as `image, scale, blur, threshold, morph_kernel_size, ros_stuff` and draws the strokes between the different nodes or keypoints identified. As part of the function, the first step involved is the pre-processing of the image. Code snippets below describe how the function has been built.

```
_, _, _, skeleton = preview_skeleton(
    image, scale, blur, threshold, morph_kernel_size
)
```

This snippet performs the following functions:

- **scale**: resizes the image, reducing noise and computation.
- **blur**: smooth out pixel-level noise before thresholding
- **threshold**: distinguish between brighter and darker pixels for distinguishing between background and drawing subject. In the snippet, pixels brighter than 190 (out of 255) are treated as background and darker pixels become the drawing subject
- **morph_kernel_size**: an operation to help clean up isolated specks after thresholding

Next, the stroke end points and junctions are identified. The premise for this is based on the connectivity of the strokes. An endpoint is where two strokes meet, while a junction is where three strokes meet. With this, the required strokes is extracted from the skeleton derived from the snippet above and the nearby strokes are merged. A snippet for this operation is shown below.

```
_, _, _, endpoints, _, junctions, _ = build_pixel_graph(
    skeleton)
strokes = extract_strokes_from_skeleton(skeleton)
strokes = merge_nearby_strokes(strokes, 8.0)
```

For merging, the number in the function's argument is a measure of the pixel range within which the strokes are merged. For example, an 8, as is passed in the given snippet, means that strokes with endpoint within 8 pixels in length would be merged together.
Finally, the strokes are mapped into the world-frame. They are also mapped onto a 280×215mm canvas and redundant points closer than 10mm apart are removed, as shown in the snippet below.

```
strokes_w = strokes_to_world(strokes, width, height, (135,3)
    , 280, 215, margin_mm=20)
r_strokes = reduce_stroke_points(strokes_w, 10)
```

2.2 Robot Execution

This execution involves concepts used over the whole semester. This first uses the inverse kinematics function derived in earlier labs to find the joint angles for every stroke's start position. Next, these joint angles are passed to forward kinematics functions and as a result, the robot moves the pen (at the end effector) to the stroke's starting point. Before touching down to the canvas or paper, two configurations are set based on the height above the required end effector position. These are 'PEN UP' and "PEN DOWN', respectively, being the heights at which the robot can move the pen freely and the height at which the image is drawn on the canvas.

```
PEN_UP = 60   # safe travel height (mm)
PEN_DOWN = 12 # drawing height (mm)
```

```
start = stroke[0]
start_thetas = lab_invk(start[0], start[1], PEN_UP, 0)
move_arm(ros_stuff["pub_cmd"], ros_stuff["loop_rate"],
         start_thetas, ros_stuff["vel"], ros_stuff["accel"],
             "J")
```

The above snippet, just controls the first stroke in the picture, while the snippet below iterates over all the strokes. This shows the general steps of the execution as described above - first identifying the joint angles and then using them to move to the desired final location.

```
for pos in stroke:
    thetas = lab_invk(pos[0], pos[1], PEN_DOWN, 0)
    move_arm(ros_stuff["pub_cmd"], ros_stuff["loop_rate"],
             thetas, ros_stuff["vel"], ros_stuff["accel"], "
                 J")
```

This guarantees the arm fully reaches each point before moving to the next, giving the drawing accuracy at the cost of speed.

```
if( abs(thetas[0]-driver_msg.destination[0]) < 0.0005 and \
```

```
    abs(thetas[1]-driver_msg.destination[1]) < 0.0005 and \
    ...
    abs(thetas[5]-driver_msg.destination[5]) < 0.0005 ):
    at_goal = 1
```

3 Conclusion and Discussion

In conclusion, all the concepts learned throughout the semester have been used to draw a given image onto a canvas. A function has been defined to trace out the strokes onto a canvas and another one has been defined to move the end effector with the pen to the required starting point of the stroke.
The approach was evaluated by how close the UR3 robot could copy the actual given image. Throughout the different trials, specific errors seen were in the distinguishing of the background from the actual image. More trials and tuning four threshold values (see section 2) helped give a better and closer image to the actual one.
The labs were very enjoyable and taught important concepts thoroughly.